🗺️ PUBG Map Overlay: The Absolute Beginner's Master Guide

Welcome to the ultimate deep-dive into the PUBG Map Simulator. This guide is designed for developers who are new to game math, high-frequency rendering, and HTML5 Canvas.

Think of this entire application as a **Translation Engine**. Its only job is to take raw, invisible numbers (PCOB Data) and translate them into a breathing, moving map on your screen.

🏗️ Part 1: The Architecture (React State vs. Refs)

In a normal website, you use `useState` to update text or colors. However, standard React state is too slow for games. If you try to update 64 players 60 times a second using `useState`, React will try to re-calculate the entire website 60 times a second, causing "lag" or a browser crash.

To solve this, we use the **Ref Pattern**:

1. `simulatorState` **(The UI Controller):** Standard React state. It only handles the sliders on the left.
2. `gameStateRef` **(The Source of Truth):** A React `useRef`. This is a "private box" that holds the latest data from the server. Changes here do NOT trigger a screen refresh.
3. `renderStateRef` **(The Visual Interpolator):** This tracks where the player *looks like* they are. This is the secret to smooth movement. The "Truth" might say a player is at X=500, but the "Visual" state is currently at X=490 and sliding toward 500.

🚂 Part 2: The Execution Flow (What calls what?)

Here is the step-by-step lifecycle of the application:

1. **Mounting:** The component loads.
2. **Pre-loading (** `useEffect #1` **):** We download all map PNGs into `imagesRef.current`. We use `img.crossOrigin = "Anonymous"` to ensure the Canvas has permission to read the image pixels.
3. **The Engine Start (** `useEffect #2` **):** We call `requestAnimationFrame(renderLoop)`. This is a high-priority browser signal that synchronizes our code with your monitor's refresh rate (60Hz).
4. **The Loop:**
   - `renderLoop` runs.
   - It calculates math for that specific millisecond.
   - It draws the frame.

- At the very bottom, it calls `requestAnimationFrame(renderLoop)` again.
- **This creates an infinite loop.**

📏 Part 3: The Scaling Math (Raw X/Y to Pixels)

PCOB sends raw coordinates. For Erangel, the world is `800,000` units wide. Our screen is only `1080` pixels. We need to find a **Universal Map Ratio**.

The Math:

$$Scale = \frac{ScreenPixels}{TotalGameUnits}$$

Real-World Example (Erangel):

If a player is at **X: 400,000** (Dead center of the 800k map):

1. **Find the Scale:** $1080/800,000 = 0.00135$.
2. **Multiply:** $400,000 \times 0.00135 = 540$ pixels.
3. The dot is drawn at exactly **540 pixels**, which is the dead center of your screen.

Real-World Example (Sanhok):

Sanhok is only **400,000** units wide. If a player is at **X: 200,000** (Dead center):

1. **Find the Scale:** $1080/400,000 = 0.0027$.
2. **Multiply:** $200,000 \times 0.0027 = 540$ pixels.
3. **Accuracy Check:** In both cases, the math correctly identified that the player is in the middle of the world and placed them in the middle of your screen.

🪄 Part 4: The `drawZone` Function (Robotic Pen)

We use this function to draw the circles. Here is exactly what the Canvas API commands are doing:

```
const drawZone = (visualZone, strokeColor, fillColor, lineWidth) => {
  // 1. Reset the "Pen". Tells the canvas a new shape is starting.
  ctx.beginPath();

  // 2. Apply the Scaling Math to the X, Y, and Radius
  const pixelX = visualZone.x * scale;
  const pixelY = visualZone.y * scale;
  const pixelRadius = visualZone.radius * scale;

  // 3. Define the Circle path: (centerX, centerY, radius, startAngle, endAngle)
```

```
    // 2 * Math.PI represents a full 360-degree rotation in Radians.
    ctx.arc(pixelX, pixelY, pixelRadius, 0, 2 * Math.PI);

    // 4. Set "Marker" properties
    ctx.lineWidth = lineWidth;
    ctx.strokeStyle = strokeColor;

    // 5. If we have a fill (like the blue zone), pour the paint inside
    if (fillColor) {
      ctx.fillStyle = fillColor;
      ctx.fill();
    }

    // 6. Trace the outline onto the paper
    ctx.stroke();
  };
```

🐸 Part 5: Player Smoothing (Asymptotic Lerping)

WebSocket data is jerky. It updates every 1 second. Without math, players would "teleport" once a second. We use **Asymptotic Lerping** to fill the gaps.

The Analogy: The Frog on the Lily Pad Imagine a frog trying to reach a lily pad 100 feet away. Every frame (1/60th of a second), the frog only jumps **10% of the remaining distance**.

- **Frame 1:** 100ft away $\longrightarrow$ Jumps 10ft (90ft left).
- **Frame 2:** 90ft away $\longrightarrow$ Jumps 9ft (81ft left).
- **Frame 3:** 81ft away $\longrightarrow$ Jumps 8.1ft...

This creates a smooth slide that starts fast and slows down as it reaches the target. On your screen, this looks like a natural, gliding movement.

The Code Logic:

```
// CurrentPos = CurrentPos + (DistanceToTarget * 0.1)
visualPos.x += (player.location.x - visualPos.x) * 0.1;
visualPos.y += (player.location.y - visualPos.y) * 0.1;
```

🔮 Part 6: Blue Zone Shrinking (Time-Based Lerping)

The Blue Zone must move at a **constant speed** over a fixed time (e.g., 15 seconds). It is not allowed to slow down like the frog.

The Analogy: The Train Schedule If a train leaves Station A at 1:00 PM and arrives at Station B at 2:00 PM, we know that at 1:30 PM (50% of the time), the train must be at exactly 50% of the distance.

The Math Steps:

1. **Find Elapsed Time:** `CurrentTime - StartTime` .
2. **Calculate Progress (0 to 1):** `ElapsedTime / TotalDuration` .
3. **Apply to Position:** `StartPosition + (TotalDistance * Progress)` .

The Code Implementation:

```
// 1. Calculate percentage of time passed (e.g. 0.5 for 50%)
const progress = Math.min(elapsed / anim.duration, 1.0);

// 2. Move X, Y, and Radius by that exact percentage
// NewPos = StartPos + (DifferenceBetweenTargetAndStart * Progress)
renderPosMap.zones.BlueZone.x = anim.startX + (anim.targetX - anim.startX) * proc
renderPosMap.zones.BlueZone.y = anim.startY + (anim.targetY - anim.startY) * proc
renderPosMap.zones.BlueZone.radius = anim.startRadius + (anim.targetRadius - anim
```

🌍 Part 7: Handling Different Map Sizes (8x8, 6x6, 4x4)

You asked if the positions will stay accurate on different sized maps. **Yes.**

Because we calculate the `scale` every single frame based on the `MAPS[state.mapType].size` , the math adapts automatically:

8x8km Map (Erangel/Miramar):

- **Total Units:** 800,000
- **Scale:** $1080/800,000 = 0.00135$
- A player at **200,000** (25% of map) results in: $200,000 \times 0.00135 = 270$ pixels.

4x4km Map (Sanhok):

- **Total Units:** 400,000
- **Scale:** $1080/400,000 = 0.0027$
- A player at **100,000** (25% of map) results in: $100,000 \times 0.0027 = 270$ pixels.

Why this is "Pinpoint" Accurate:

In both examples, the player was at the **25% mark** of their respective worlds. The math correctly calculated the different scales to ensure both players landed at exactly **270 pixels** on